Multi-Agent Retrieval for Learning-State-Aware Educational Tutoring

Anonymous Authors

Abstract— Large language models can generate fluent tutoring responses, but they need grounding that respects both instructional evidence and the learner's current state. Classical retrieval-augmented generation (RAG) grounds generation through a fixed retrieve-then-read controller: the query is matched against a corpus, and the generator reads the returned passages. That design is often insufficient for tutoring, where the same surface question can require different evidence, scaffolding, or rubric constraints for different learners. We propose Multi-Agent Retrieval, a learning-state-aware framework that places a supervisory router, planner, diagnoser, rubric module, retriever, tutor, and critic around a fixed downstream tutor. The framework first builds a tutoring brief from the sample, inferred learning state, and answer criteria; retrieval is then invoked only when the route indicates that external evidence is needed. We validate the framework against classical RAG under matched backbones, corpus, reranker, and response budget on TutorEval and EduBench. The proposed framework improves fixed-tutor quality metrics over classical RAG while using fewer tokens; on EduBench it also reduces latency and backbone-call count. These results support the framework claim: for educational tutoring, grounding should be controlled by pedagogical state and task regime, not by the surface query alone.

I. Introduction

Large language models (LLMs) are increasingly used as educational tutors because they can generate explanations, answer questions, diagnose errors, and provide feedback in natural language. However, an LLM tutor cannot rely only on its parametric knowledge. Educational responses often need to be grounded in specific instructional materials, such as textbook chapters, course notes, rubrics, worked examples, or learner profiles. Without such grounding, the tutor may produce fluent responses that are misaligned with the curriculum, learning objectives, or student context.

Classical retrieval-augmented generation (RAG) has therefore become the default mechanism for grounding LLM-based tutoring [1]. In classical RAG, the tutor retrieves evidence primarily from the query representation: the input query is embedded, the top-k most similar educational passages are retrieved, and the LLM generates a response conditioned on those passages. This retrieve-then-read pattern is effective when the query itself determines the needed evidence, but tutoring often requires more than topical similarity.

Educational tutoring is more complex than factual question answering. A student's question is often only a partial signal of what the tutor needs to know. The appropriate response may depend on the learner's current level, goals, weak points, recent confusion, prior dialogue, and likely misconceptions. We refer to this student-specific information as the learner's personalized learning state. For example, the query "How do I solve an ordinary differential equation?" should trigger different context for a high-school student encountering slope fields than for a college student learning separation of variables or Laplace transforms. A query-only RAG system retrieves topic-similar ODE passages unless the learner state is manually appended to the query.

Appending learner state, dialogue history, grading criteria, and task constraints directly to the retrieval query is not a principled solution. It shifts the burden onto a single query embedding that must compress heterogeneous information: the topic, learner level, suspected misconception, instructional goal, and response constraints. This can blur search intent and produce evidence that satisfies some needs while missing others. Once such state is manually appended and curated, the system is no longer the classical retrieve-then-read controller.

We propose Multi-Agent Retrieval as a framework for this setting. Instead of letting the retriever act as the top-level controller, the framework uses a supervisory router to identify the tutoring regime and decide whether retrieval is needed. A planner scopes the search intent, a diagnoser summarizes visible learning state, a rubric module states what the answer must satisfy, and the retriever becomes a conditional tool rather than a mandatory first step. The downstream tutor and critic remain fixed, so the framework's contribution is the structure that constrains what context reaches the tutor.

The validation question is whether this framework improves tutoring behavior under fair infrastructure. We therefore compare the proposed framework with classical RAG while holding the backbone, corpus, reranker, response cap, tutor, and critic fixed. This comparison has an important limitation: the classical baseline does not receive the same agent-built tutoring brief, because that brief is part of the proposed framework. The result should therefore be read as a validation of the framework's structural advantage, not as a claim that retrieval alone is obsolete.

Educational tutoring includes explanation, misconception diagnosis, rubric-based scoring, and next-step feedback. These subproblems require different grounding

Submitted to the 2026 IEEE International Conference on Systems, Man, and Cybernetics (SMC). Single-blind review.

signals, so retrieval recall alone is not enough [2], [3]. Our validation on TutorEval and EduBench shows that the same tutor scores higher when fed context assembled by Multi-Agent Retrieval; on EduBench it also reduces latency and backbone calls.

Contributions. (i) We propose a learning-state-aware multi-agent retrieval framework for educational tutoring, with an explicit supervisory router, planner, diagnoser, rubric module, conditional retriever, tutor, and critic. (ii) We formalize the router and retrieval gate as transparent, training-free controls that determine when learner state, rubric constraints, and external evidence should enter the tutoring brief. (iii) We validate the framework against classical RAG on two educational tutoring benchmarks under shared backbone, corpus, reranker, and response budget, reporting both tutoring quality and resource trade-offs.

II. Related Work

Classical RAG fixes retrieval as the first controller: the query is matched to passages, and generation conditions on the returned evidence [1]. Selective and corrective RAG methods improve this controller. Self-RAG learns when to retrieve, generate, and critique; CRAG evaluates retrieved documents and attempts correction when retrieval is unreliable; Adaptive-RAG and TARG vary retrieval behavior by query complexity or retrieval utility [4]–[7]. These methods address an evidence-control problem: whether the passages retrieved for a query are necessary, sufficient, or reliable. They do not address the tutoring-control problem that motivates this paper: before retrieval, the system must decide what the learner appears to need, which rubric constraints matter, and whether external evidence is even the right source of grounding.

Agentic RAG and multi-agent LLM systems move beyond a single retrieve-then-read step. ReAct and AutoGen show that tool use and role decomposition can improve complex task execution [8], [9]. MA-RAG uses multiple agents for query disambiguation, evidence extraction, and answer synthesis on ambiguous or multi-hop QA tasks [10]. These systems are related in their use of specialist roles, but they are not designed around educational tutoring. They do not define pedagogical regimes, infer visible learner state, or bind the final response to rubric-style answer criteria. Comparing directly against every general agentic framework would therefore test a broad agent platform question; our validation instead isolates the proposed tutoring-specific controller against classical RAG under shared infrastructure.

Recent educational work reinforces why this tutoring-specific control is needed. AgentTutor builds a multi-turn personalized learning system with learner assessment, strategy selection, reflection, and memory [11]; adaptivity studies show that LLM tutors are sensitive to omitted student-context features and still struggle to match the adaptivity of intelligent tutoring systems [12]; tutoring benchmarks emphasize pedagogy, mistake diagnosis, guidance quality, and learner support rather than short-form answer accuracy alone [2], [3], [13], [14]. These works motivate the educational criteria, but they do not formulate retrieval as a learner-state-aware control problem. Our contribution is therefore not another benchmark comparison or a generic multi-agent platform; it is a framework that makes learner-state diagnosis, rubric constraints, search intent, retrieval gating, and tutor/critic generation explicit before the final tutor writes.

III. Methodology

Our proposed framework, Multi-Agent Retrieval, is a tutoring pipeline organized around a supervisory controller. For each sample, the controller reads the question, dialogue, context text, rubric items, and metadata, then assigns a pedagogical regime and decides which modules should run. The hybrid pipeline then builds a tutoring brief before answer generation: the planner states the instructional goal and search intent, the diagnoser summarizes visible learning state, the rubric module lists answer criteria, and retrieval is invoked only when external evidence is needed. Fig. 1 sketches this pipeline.

This separation matters because tutoring failures arise from different sources. Sometimes the missing input is external evidence, such as the relevant paragraph in a long chapter. Sometimes it is pedagogical state, such as whether the learner needs a first-principles explanation, a misconception correction, or a rubric-aligned score. A retrieve-first controller treats these cases too similarly. Our framework makes the tutoring brief explicit before answer generation.

The design principle is simple: retrieve when external evidence is genuinely needed, and otherwise spend the context budget on the learner state and answer criteria. Validation keeps the tutor/critic backbone, corpus, reranker, packer, and response budget fixed so that the classical RAG comparison tests how context is assembled before the same tutor writes. Backbone calls and tokens are measured as resource outcomes, not forced to be equal.

A. Supervisory Routing

The supervisor is the framework's decision-making component. It estimates whether a sample primarily needs external evidence, pedagogical coordination, rubric feedback, planning, or dialogue-sensitive tutoring. The reported runs use transparent heuristics rather than a trained router. For an adapter-normalized sample x, the router first builds a normalized text blob $b(x)$ from the question, optional context, rubric text, and dialogue.

The first routing quantity is a cue-hit rate. For each cue phrase u, define $m(u, x) = 1$ if u appears in $b(x)$, and

TABLE I

Framework components and responsibilities.

Component	Responsibility
Supervisor	Assigns pedagogical regime and activates retrieval, rubric, and critic switches.
Planner	Converts the tutoring request into an operating plan and up to three scoped retrieval queries.
Diagnoser	Infers visible learning state from the prompt and dialogue, including level and likely misconceptions.
Rubric module	Summarizes explicit rubric items or default answer-quality criteria.
Retriever	Runs only when requested by the route or fallback gate, then reranks and packs evidence.
Tutor/Critic	Generates the final response and checks evidence markers, rubric coverage, and pedagogy.

$m(u, x) = 0$ otherwise. For a cue family G, the router computes

$$\kappa_G(x) = \frac{\sum_{u \in G} m(u, x)}{|G|}, \tag{1}$$

where $|G|$ is the number of cue phrases in that family. Thus $\kappa_G(x)$ is just the fraction of cues that were observed: 0 means no cue matched, and 1 means every cue matched. The cue families are evidence (E), coordination (C), rubric (R), planning (P), and tutoring (T).

The cue-hit rate is then converted into a routing score:

$$s_j(x) = \min\{1, \max\{0, \kappa_j(x) + B_j(x) + P_j(d_x)\}\}, \tag{2}$$

where $j \in \{e, c, r, p, t\}$ corresponds to the five families above. The term $B_j(x)$ is a field bonus from visible sample structure: context or image fields increase the evidence score, rubric fields increase the rubric score, and longer dialogue increases the tutoring score. The term $P_j(d_x)$ is a dataset prior from the adapter label d_x; for example, evidence-grounded datasets receive an evidence prior. The outer min / max only keeps the score between 0 and 1. These scores are not probabilities; they are transparent control signals used to decide which modules should run.

The router then assigns a pedagogical regime: lesson planning, rubric feedback, adaptive tutoring, or evidence-grounded reasoning. If the adapter supplies a regime hint, the supervisor uses it; otherwise it chooses the regime with the strongest corresponding score subject to fixed thresholds. Retrieval is controlled by the binary gate

$$g(x) = \begin{cases} 1, & s_e(x) \geq 0.35 \text{ or } R(x) = \text{EG}, \\ 0, & \text{otherwise.} \end{cases} \tag{3}$$

Here $s_e(x)$ is the evidence score and $R(x) = \text{EG}$ means evidence-grounded reasoning. In words, retrieval is requested when the sample looks evidence-heavy or the regime itself is evidence-grounded. If $g(x) = 0$, the framework still builds the tutoring brief but skips retrieval. If no chunks have been retrieved and the evidence score is at least 0.45, one fallback retrieval pass is issued using the raw question. Thus the router constrains both how much pedagogical coordination to activate and whether the retriever should be used.

B. Specialist agents and shared context

All implemented mechanisms operate over the same tutoring prompt, inferred learning state, rubric summary, retrieved evidence, and draft answer. The tutor and the critic are held constant across all implemented mechanisms: the tutor writes from the supplied context, and the route-controlled critic may revise the draft; on TutorEval and EduBench it is usually enabled because both adapters stamp adaptive-tutoring regime hints. The only moving parts are the upstream context builders. In the released paper configuration, planner, diagnoser, and rubric are deterministic heuristics. Their roles are asymmetric. The planner alone is retrieval-facing: it emits up to three deduplicated queries from the raw question, extracted key terms, dataset cueing, and the first rubric items. The diagnoser and rubric agent are tutor-facing: they summarize transient learning state and answer criteria for the tutor prompt, but they do not alter retriever scoring. In this release, learning state is inferred from the current question and dialogue rather than from a persistent profile store.

The tutor itself is dataset-aware but architecturally fixed. Its prompt includes the dataset name, question, recent dialogue, inferred learning state, operating plan, rubric summary, and either retrieved evidence or inline benchmark context. It is instructed to be correct, pedagogically useful, concise, and to end with a next step or check-for-understanding when appropriate. Retrieved evidence must be cited with [doc_id] markers. EduBench consensus rows require a JSON object with a score, scoring details, and personalized feedback; TutorEval and the other tutoring-style profiles use instructional prose. This design isolates the effect of changing the retrieval context while leaving the downstream writer unchanged; personalization reaches retrieval mainly through planner query formulation, not through profile-aware reranking.

C. Hybrid retrieval, reranking, and context packing

The retrieval stack is intentionally lightweight and also follows the codebase exactly. The following equations are scoring rules, not learned models: a larger score means the chunk is ranked higher for the current query. For a query q and candidate chunk c, the hybrid index first computes

$$\begin{aligned} \text{score}_0 = 0.58\, k_w + 0.17\, k_{ch} \\ + 0.25\, k_\ell + \beta, \end{aligned} \tag{4}$$

where all terms are computed for the current (q, c) pair. Here k_w is word-level TF–IDF similarity, k_{ch} is character 3–5 gram TF–IDF similarity, and k_ℓ is the similarity in the truncated-SVD latent space. The weights state how much the released retriever trusts each signal: word similarity contributes most, latent semantic similarity is second, and character similarity helps with spelling variants and short phrases. The query boost β adds 0.01 per overlapping title token and, in citation-seeking

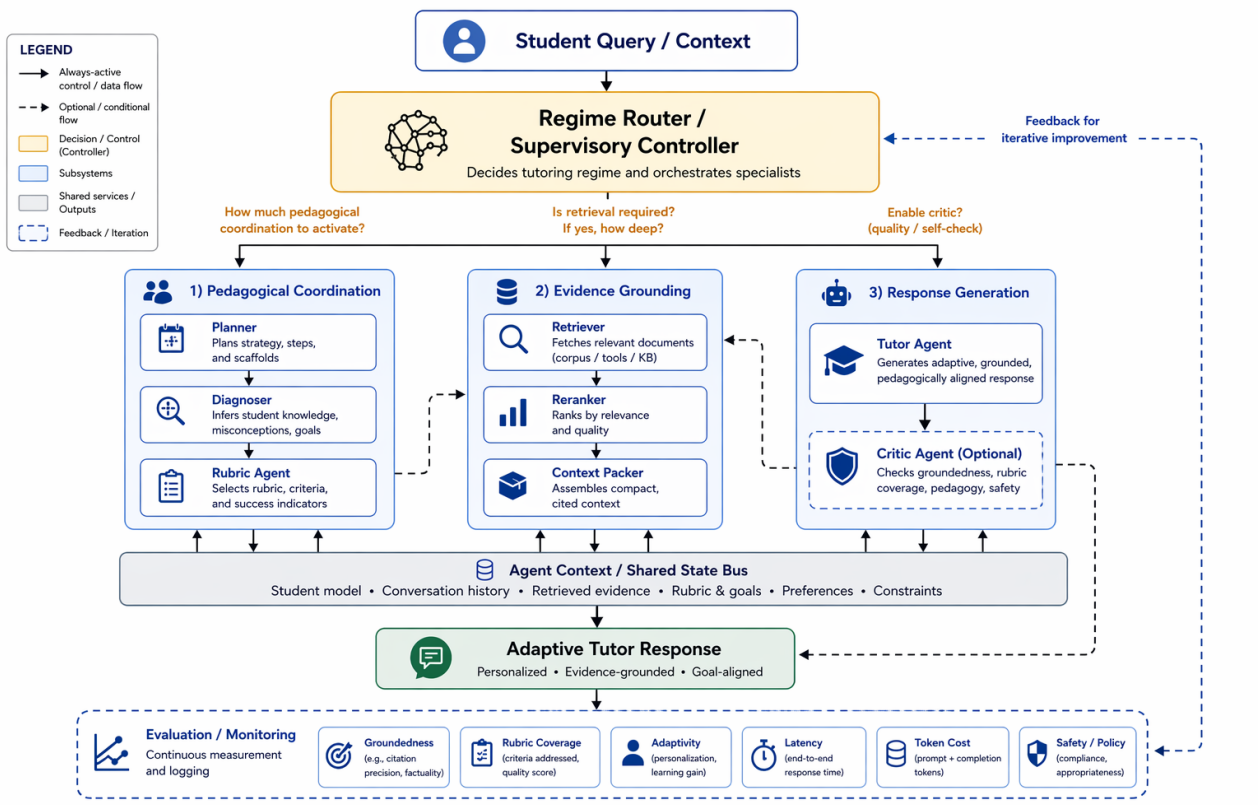

Fig. 1. Main framework of Multi-Agent Retrieval. The supervisory router scores the tutoring sample, assigns the pedagogical regime, and decides whether retrieval is needed. Planner, diagnoser, and rubric modules assemble the tutoring brief before the fixed tutor writes the final response.

mode, an additional 0.025 for chunks whose source type is reference, lecture, or document. When multiple planner queries are issued, the retriever merges all returned candidates and deduplicates them by chunk identifier before reranking.

In the released paper configuration, no trained reranker is loaded, so the heuristic reranker score is

$$\begin{aligned}\text{score}_1 = 0.55\,\text{score}_0 + 0.20\,o_t + 0.10\,o_h \\ + 0.06\,m_p + 0.05\,o_n + 0.04\,\rho,\end{aligned} \tag{5}$$

where all terms are again computed for the current (q, c) pair. The variables o_t, o_h, m_p, and o_n denote token overlap, title overlap, exact phrase-match score, and numeric overlap, and ρ is a brevity prior. This reranker favors chunks that match the question text, title, exact phrases, and numbers while mildly preferring concise chunks. Once planner queries are merged, reranking remains question-centric and does not consume the learner summary directly.

Final context packing balances relevance against redundancy:

$$c^\star = \arg\max_{c \in \mathcal{C} \setminus S} \lambda\,\text{score}_1(q, c) - (1-\lambda)\max_{c' \in S} \text{overlap}(c, c'), \tag{6}$$

where $c^\star$ is the next selected chunk, $\mathcal{C}$ is the reranked candidate pool, S is the set of already packed chunks, and $\text{overlap}(c, c')$ is the redundancy term between two chunks. The first term rewards relevance to the query; the second term penalizes selecting another chunk that repeats content already in S. With $\lambda = 0.68$, a four-chunk cap, and a 6000-character context budget, this keeps the comparison focused on retrieval quality rather than on simply enlarging the context window.

D. Evaluation protocol

Evaluation is dataset-aware, but the reported metrics are internal automatic proxies implemented in the released evaluator rather than official benchmark scoring scripts. The headline table's composite score (Comp.) is a paper-level weighted summary from the released analysis pipeline, reported only as a convenience overview; we interpret the task-specific metrics directly. Token-F1 is the token-overlap F1 between the model answer and the benchmark reference answer. Rubric coverage is the fraction of rubric items the answer covers, using either near-verbatim phrase match or sufficiently strong token overlap anchored by a content-bearing token. Latency, total tokens, and backbone-call count capture inference cost, and backbone calls correspond to the released system's llm_call_count.

For EduBench, we additionally report EB12D, the mean over 12 rubric-oriented signals spanning scenario adaptation, factual reasoning, and pedagogical application. Those signals reward output-format compliance, supportive but non-negative tone, relevance to the

question, use of scenario elements from the sample, alignment with the benchmark score, domain-knowledge grounding, reasoning quality, error identification, clarity, motivational guidance, personalization, and higher-order prompting. We report EduAlign as $1 - |s(y) - \bar{s}|/100$, where y is the model's JSON output, $s(y)$ is the numeric score extracted from it, and $\bar{s}$ is the benchmark reference score.

For TutorEval, we report tutor-hit as the mean key-point credit over the K expected teaching points, with full credit for explicit coverage, partial credit for moderate token overlap, and zero otherwise. We also track a chapter-grounding proxy by measuring whether the response stays aligned with retrieved evidence when retrieval is used, or with the inline chapter text otherwise. Concretely, EduBench is evaluated as a structured educational-judgment task, whereas TutorEval is evaluated as a chapter-grounded tutoring-response task. In the per-example evaluator outputs, off-profile metrics remain present as zeros; in the paper tables, benchmark-inapplicable columns are simply not shown. The released code also records a workload audit $U = T + 160Q_r + 90C_r + 240N_a + 30E_v$, where T is total tokens, Q_r retrieval queries, C_r retrieved chunks, N_a materialized agent outputs, and E_v trace events. This score is used only as an efficiency audit, not as a training objective. A retrieval-agnostic corpus-factuality check is supported when a compatible shared index is available, but the archived main comparison predates that audit, so we do not present it as a headline result here.

IV. Validation Experiments

We validate the framework on two public educational benchmarks. TutorEval [13], released with LM-Science-Tutor, contains 834 expert-written science tutoring questions in its open-book setting. Each question is paired with a long STEM chapter and expert teaching key points, so the system must explain, disambiguate, and address likely misconceptions while staying chapter-grounded. EduBench [14] covers nine educational scenarios across student- and teacher-oriented tasks, including question answering, error correction, personalized support, emotional support, grading, teaching-material generation, and personalized content creation. Its rows contain prompts, scenario metadata, and reference model predictions; our adapter converts those references into consensus-style supervision through score statistics and rubric terms. For reporting, we use the 1562-example four-way EduBench intersection completed by all forced architectures.

The primary validation compares classical RAG with Multi-Agent Retrieval under matched Qwen3.5-4B infrastructure, reasoning budget 512, temperature 0.15, response cap 900, shared corpus, shared reranker, and fixed retrieval hyperparameters. Classical RAG retrieves before tutoring and does not receive the agent-built learning-state and rubric brief. This is the intended contrast: the baseline tests retrieve-then-read grounding, while our framework tests supervised context construction plus conditional retrieval.

TABLE II

TutorEval quality validation (n=834). Higher is better.

Method	Comp.	Tok-F1	TE Hit	Rubric
Classical RAG	0.377	0.112	0.938	0.919
Ours	0.383	0.115	0.957	0.944
Δ	+0.006	$+0.004^{\star}$	$+0.019^{\star\star}$	$+0.026^{\star\star}$

TABLE III

EduBench quality validation (n=1562). Higher is better.

Method	EB12D	Rubric	EduAlign
Classical RAG	0.367	0.705	0.166
Ours	0.398	0.891	0.126
Δ	$+0.031^{\star\star\star}$	$+0.186^{\star\star\star}$	$-0.040^{\star\star}$

$^{\star\star\star}p<10^{-4}$, $^{\star\star}p<0.01$, $^{\star}p<0.05$; CIs use 2000 paired bootstraps and p-values use 10000 permutations.

The validation is fair at the infrastructure level but intentionally not identical at the context-construction level. Giving classical RAG the planner, diagnoser, and rubric outputs would convert it into the proposed framework. Conversely, withholding those outputs from Multi-Agent Retrieval would remove the claimed contribution. We therefore report quality and resource outcomes together: a useful framework should improve tutoring behavior without hiding an unacceptable token, latency, or backbone-call cost.

A. Framework validation

Tables II and III report paired matched-sample comparisons with bootstrap confidence intervals and permutation p-values. On TutorEval, Multi-Agent Retrieval raises Token-F1 by +0.004, tutor-hit by +0.019, and rubric coverage by +0.026, while using 484 fewer tokens. Latency trends lower but is not reliable (p=0.21). Because the forced TutorEval harness computes route metadata before architecture forcing, the proposed framework retrieves on every TutorEval sample; this result therefore tests whether planning, diagnosis, and rubric context help even when chapter evidence is always consulted.

B. Trade-offs and ablations

On TutorEval, the 95% confidence intervals are $[0.0003, 0.007]$ for Token-F1, $[0.006, 0.032]$ for tutor-hit, and $[0.007, 0.045]$ for rubric coverage. The framework also reduces average latency from 51 903 to 49 113 ms, tokens from 3 975 to 3 490, and backbone calls from 1.84 to 1.56.

On the EduBench intersection, multi-agent retrieval improves EB12D by +0.031 and rubric coverage by +0.186, while reducing latency by 5,923 ms, tokens by 1,794, and backbone calls by 0.49. Classical RAG keeps a small EduAlign edge, but the frozen hybrid skips retrieval on every EduBench sample, so these prompts are usually better served by learner- and rubric-centered coordination than by external lookup.

For EduBench, the 95% confidence intervals are $[0.027, 0.036]$ for EB12D, $[0.174, 0.199]$ for rubric coverage, and $[-0.069, -0.011]$ for EduAlign. The cost intervals are also favorable: latency $[-6628, -5242]$ ms, tokens $[-1845, -1738]$, and calls $[-0.52, -0.46]$.

The framework can use more modules than classical RAG, so tokens and calls must be measured rather than assumed. In these runs, scoped planner queries and the retrieval gate reduce total token use rather than increasing it. Retrieval-free diagnostic runs confirm that removing corpus evidence is not a meaningful main baseline for TutorEval, where no-retrieval variants trail chapter-grounded methods. Two executed 4B ablations on a shared TutorEval slice (n=100) further isolate the mechanism: forcing retrieval hurts Token-F1 (-0.003), tutor-hit (-0.015), and rubric coverage (-0.025), while disabling the shared critic shows that the main gain does not come from critic revision. On a pinned Qwen3.5-27B-FP8 backbone, TutorEval deltas collapse near zero, but hybrid remains more efficient; EduBench 27B repeats the pattern with EB12D $+0.032$, rubric $+0.210$, tokens $-1{,}861$, and latency $-22{,}070$ ms.

These results clarify the intended use of the framework. Multi-Agent Retrieval is not a universal replacement for retrieval; it is a controller for deciding what kind of grounding a tutoring sample needs. When the sample is chapter-grounded, retrieval remains important, but the planner and diagnoser make the retrieved material easier for the tutor to use pedagogically. When the sample already contains learner profile, rubric, or scenario constraints, mandatory retrieval can add irrelevant topical text and consume tokens that are better spent on learner-state and answer-criteria summaries. The framework therefore gives reviewers an inspectable decision path: which regime was assigned, why retrieval was or was not requested, which criteria were passed to the tutor, and how the critic checked the draft. This traceability is important in educational settings, where a correct answer may still be poor tutoring if it ignores the learner's stage or grading context.

V. Limitations

The study validates a framework rather than a final tutoring product. It fixes backbone, cap, corpus, and retriever but not the full system structure; it reports automatic rather than human evaluation; and it is limited to two open-weight backbones, one shared corpus per benchmark, English-only data, and benchmark outputs rather than downstream learning gains.

VI. Conclusion

Multi-Agent Retrieval reframes grounding for educational tutoring as supervised context construction rather than query-only passage lookup. Under shared infrastructure, it improves fixed-tutor quality and efficiency over classical RAG, showing that learner state, rubric constraints, and conditional evidence should be coordinated before the tutor writes.

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. K"uttler, M. Lewis, W. tau Yih, T. Rockt"aschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459–9474. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[2] J. Macina, N. Daheim, I. Hakimi, M. Kapur, I. Gurevych, and M. Sachan, "Mathtutorbench: A benchmark for measuring open-ended pedagogical capabilities of LLM tutors," in Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing. Suzhou, China: Association for Computational Linguistics, 2025, pp. 204–221. [Online]. Available: https://aclanthology.org/2025.emnlp-main.11/

[3] R. S. Srinivasa, Z. Che, C. B. C. Zhang, D. Mares, E. Hernandez, J. Park, D. Lee, G. Mangialardi, C. Ng, E.-Y. H. Cardona, A. Gunjal, Y. He, B. Liu, and C. Xing, "Tutorbench: A benchmark to assess tutoring capabilities of large language models," 2025. [Online]. Available: https://arxiv.org/abs/2510.02663

[4] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in International Conference on Learning Representations, 2024. [Online]. Available: https://arxiv.org/abs/2310.11511

[5] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective retrieval augmented generation," 2024. [Online]. Available: https://arxiv.org/abs/2401.15884

[6] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, "Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity," in Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL), 2024. [Online]. Available: https://arxiv.org/abs/2403.14403

[7] Y. Wang, L. Wei, and H. Ling, "Retrieval as a decision: Training-free adaptive gating for efficient RAG," 2026. [Online]. Available: https://arxiv.org/abs/2511.09803

[8] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in International Conference on Learning Representations, 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[9] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," 2024. [Online]. Available: https://arxiv.org/abs/2308.08155

[10] T. Nguyen, P. Chin, and Y.-W. Tai, "Ma-rag: Multi-agent retrieval-augmented generation via collaborative chain-of-thought reasoning," 2025. [Online]. Available: https://arxiv.org/abs/2505.20096

[11] Y. Liu, Z. Song, J. Lou, C. Wu, and J. Li, "Agenttutor: Empowering personalized learning with multi-turn interactive teaching in intelligent education systems," 2026. [Online]. Available: https://arxiv.org/abs/2601.04219

[12] C. Borchers and T. Shou, "Can large language models match tutoring system adaptivity? a benchmarking study," 2025. [Online]. Available: https://arxiv.org/abs/2504.05570

[13] A. Chevalier, J. Geng, A. Wettig, H. Chen, S. Mizera, T. Annala, M. J. Aragon, A. R. Fanlo, S. Frieder, S. Machado, A. Prabhakar, E. Thieu, J. T. Wang, Z. Wang, X. Wu, M. Xia, W. Xia, J. Yu, J.-J. Zhu, Z. J. Ren, S. Arora, and D. Chen, "Language models as science tutors," 2024. [Online]. Available: https://arxiv.org/abs/2402.11111

[14] B. Xu, Y. Bai, H. Sun, Y. Lin, S. Liu, X. Liang, Y. Li, Z. Dong, J. Zhang, Y. Deng, X. Zou, Y. Gao, and H. Huang, "Edubench: A comprehensive benchmarking dataset for evaluating large language models in diverse educational scenarios," 2025. [Online]. Available: https://arxiv.org/abs/2505.16160